

PYTHON PROGRAMMING

COMPLETE HANDBOOK — BASIC TO ADVANCED

PART 6

Module 17: NumPy

Module 18: Pandas

Module 19: Matplotlib

Continues from Parts 1-5 (Modules 1-16)

TABLE OF CONTENTS

TABLE OF CONTENTS	2
MODULE 17: NUMPY	4
17.1 Arrays	4
17.2 Indexing	4
17.3 Slicing	5
17.4 Mathematical Operations	5
17.5 Random Module	5
Data Science Example — Basic Statistics on Exam Scores	6
Interview Questions — Module 17	6
MCQs — Module 17	7
Exercises — Module 17	7
MODULE 18: PANDAS	8
18.1 Series	8
18.2 DataFrame	8
18.3 Reading CSV	8
18.4 Cleaning Data	9
18.5 Filtering	9
18.6 GroupBy	9
Business Dataset Example — Sales Report Analysis	10
Interview Questions — Module 18	10
MCQs — Module 18	11
Exercises — Module 18	11
MODULE 19: MATPLOTLIB	12
19.1 Line Chart	12
19.2 Bar Chart	12
19.3 Pie Chart	12
19.4 Histogram	13
19.5 Scatter Plot	13
Choosing the Right Chart Type	13
Visualization Project — Sales Dashboard (Multiple Subplots)	13
Interview Questions — Module 19	14
MCQs — Module 19	14
Exercises — Module 19	15
QUICK REVISION NOTES — MODULES 17-19	16
COMMON ERRORS AND FIXES — MODULES 17-19	17

CODING BEST PRACTICES — MODULES 17-19	18
--	-----------

MODULE 17: NUMPY

NumPy (Numerical Python) is the foundational library for numerical computing in Python. It introduces the `ndarray` — a fast, memory-efficient, multi-dimensional array — that performs mathematical operations much faster than plain Python lists. Almost every data science library (Pandas, scikit-learn, TensorFlow) is built on top of NumPy.

Setup

Install NumPy once with: `pip install numpy` Then import it in your script with: `import numpy as np`

17.1 Arrays

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])      # 1-D array
b = np.array([[1, 2], [3, 4]])    # 2-D array (matrix)
print(a)
print(b)
print(a.shape, b.shape)          # dimensions
```

Output:

```
[1 2 3 4 5]
[[1 2]
 [3 4]]
(5,) (2, 2)
```

NumPy arrays vs Python lists — why NumPy is faster:

- A NumPy array stores elements of the SAME data type in contiguous memory — much faster to process than a Python list, which stores generic objects scattered in memory.
- NumPy supports vectorised operations — applying an operation to an entire array at once, without writing an explicit loop.

```
lst = [1, 2, 3]
# lst * 2 would repeat the list: [1,2,3,1,2,3] — NOT elementwise!
arr = np.array([1, 2, 3])
print(arr * 2)  # elementwise multiplication
```

Output:

```
[2 4 6]
```

17.2 Indexing

```
arr = np.array([10, 20, 30, 40])
print(arr[0])      # first element
print(arr[-1])     # last element

matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix[1, 2]) # row 1, column 2
```

Output:

```
10
40
6
```

17.3 Slicing

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[1:4])      # index 1 to 3
print(arr[:3])       # first three

matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix[:, 1])  # entire column 1
print(matrix[0, :])  # entire row 0
```

Output:

```
[20 30 40]
[10 20 30]
[2 5 8]
[1 2 3]
```

17.4 Mathematical Operations

Function	Purpose	Example	Result
np.sum()	Adds all elements	np.sum(arr)	150
np.mean()	Average value	np.mean(arr)	30.0
np.max() / min()	Largest / smallest value	np.max(arr)	50
np.sqrt()	Square root, elementwise	np.sqrt([4,9])	[2. 3.]
np.sort()	Sorts the array	np.sort([3,1,2])	[1 2 3]
np.reshape()	Changes array shape	arr.reshape(2,3)	2x3 matrix

```
arr = np.array([10, 20, 30, 40, 50])
print(arr.sum(), arr.mean(), arr.max())
```

Output:

```
150 30.0 50
```

17.5 Random Module

```
np.random.seed(1)    # ensures reproducible results
print(np.random.randint(1, 100, 5))  # 5 random ints between 1-99
print(np.random.rand(3))             # 3 random floats between 0-1
```

Output:

```
[38 13 73 10 76]
[0.417 0.720 0.0001]
```

Data Science Example — Basic Statistics on Exam Scores

```
scores = np.array([78, 85, 92, 66, 74, 90, 55, 88])
print("Average:", scores.mean())
print("Highest:", scores.max())
print("Lowest:", scores.min())
print("Std Dev:", round(scores.std(), 2))
print("Above average:", scores[scores > scores.mean()])
```

Line	Explanation
1	Create a NumPy array of exam scores.
2-4	Compute average, maximum, and minimum in one call each — much simpler than manual loops.
5	Standard deviation measures how spread out the scores are.
6	Boolean indexing: <code>scores > scores.mean()</code> creates a True/False mask, and <code>scores[mask]</code> returns only the values where it's True.

Output:

```
Average: 78.5
Highest: 92
Lowest: 55
Std Dev: 12.15
Above average: [85 92 90 88]
```

Interview Questions — Module 17**Q1. Why is NumPy faster than a regular Python list for numerical work?**

Ans: NumPy arrays store elements of a single data type in contiguous memory blocks and use optimised, pre-compiled C code for operations, allowing vectorised (elementwise) computation without explicit Python-level loops — this is far faster than looping over a list.

Q2. What is broadcasting in NumPy?

Ans: Broadcasting lets NumPy perform operations between arrays of different (but compatible) shapes without explicitly copying data, e.g., adding a single number to every element of an array automatically.

Q3. What is the difference between `arr.shape` and `arr.size`?

Ans: `arr.shape` returns a tuple showing the dimensions (rows, columns). `arr.size` returns the total number of elements across all dimensions.

Q4. How do you select elements from an array that meet a condition?

Ans: Using boolean/fancy indexing: `arr[arr > 10]` returns only the elements greater than 10, by first building a True/False mask from the condition.

Q5. What does `np.reshape()` do?

Ans: It changes the shape (dimensions) of an array without changing its underlying data, as long as the total number of elements stays the same.

MCQs — Module 17

No.	Question	Answer	Options (A-D)
1	NumPy arrays are created using:	B	A) np.list() B) np.array() C) np.data() D) np.new()
2	Which attribute gives the dimensions of an array?	C	A) .size B) .len C) .shape D) .dim
3	arr * 2 on a NumPy array of [1,2,3] gives:	A	A) [2,4,6] B) [1,2,3,1,2,3] C) Error D) [1,2,3]*2
4	Which function computes the average of an array?	D	A) np.total() B) np.avg() C) np.middle() D) np.mean()
5	matrix[:, 1] selects:	B	A) Row 1 B) Column 1 (all rows) C) Element [0][1] D) Error
6	Which module generates random numbers in NumPy?	A	A) np.random B) np.rand C) np.gen D) np.rng

Exercises — Module 17

1. Create a NumPy array of 10 numbers and find its mean, median, and standard deviation.
2. Create a 3x3 identity matrix using NumPy.
3. Write a program to reshape a 1-D array of 12 elements into a 3x4 matrix.
4. Generate an array of 10 random integers between 1-100 and print only the even ones.
5. Write a program to add, subtract, and multiply two NumPy arrays elementwise.

MODULE 18: PANDAS

Pandas is Python's primary library for working with structured, tabular data — think of it as Excel inside Python, but far more powerful and scriptable. It is built on top of NumPy and is essential for any real-world data analysis task.

Setup

Install once with: `pip install pandas` Then import with: `import pandas as pd`

18.1 Series

A Series is a one-dimensional labelled array — like a single column of data, with an automatic or custom index.

```
import pandas as pd

marks = pd.Series([88, 72, 95, 60], index=["Riya", "Aman", "Neha", "Raj"])
print(marks)
print(marks["Neha"])
```

Output:

```
Riya    88
Aman    72
Neha    95
Raj     60
dtype: int64
95
```

18.2 DataFrame

A DataFrame is a two-dimensional table (rows and columns) — the core Pandas structure, similar to a spreadsheet or a SQL table.

```
data = {
    "Name": ["Riya", "Aman", "Neha"],
    "Marks": [88, 72, 95],
    "City": ["Delhi", "Pune", "Mumbai"]
}
df = pd.DataFrame(data)
print(df)
```

Output:

	Name	Marks	City
0	Riya	88	Delhi
1	Aman	72	Pune
2	Neha	95	Mumbai

18.3 Reading CSV

```
df = pd.read_csv("students.csv")
print(df.head())          # first 5 rows
print(df.info())          # column types and null counts
print(df.describe())      # statistical summary of numeric columns
```

18.4 Cleaning Data

Method	Purpose	Example	Result
isnull()	Detects missing values	df.isnull().sum()	count of NaNs per col
dropna()	Removes rows with missing values	df.dropna()	cleaned DataFrame
fillna()	Fills missing values	df.fillna(0)	NaNs replaced with 0
drop_duplicates()	Removes duplicate rows	df.drop_duplicates()	unique rows only
astype()	Converts a column's data type	df["Age"].astype(int)	column as int

```
df = pd.DataFrame({"Name": ["Riya", "Aman", None], "Marks": [88, None, 60]})
print(df.isnull().sum())
df_clean = df.fillna({"Name": "Unknown", "Marks": 0})
print(df_clean)
```

Output:

```
Name      1
Marks     1
dtype: int64
   Name  Marks
0  Riya   88.0
1  Aman    0.0
2 Unknown  60.0
```

18.5 Filtering

```
df = pd.DataFrame({"Name": ["Riya", "Aman", "Neha"], "Marks": [88, 72, 95]})

high_scorers = df[df["Marks"] > 75]
print(high_scorers)
```

Output:

```
   Name  Marks
0  Riya    88
2  Neha    95
```

18.6 GroupBy

`groupby()` splits data into groups based on a column's values, then lets you apply an aggregate function (sum, mean, count) to each group — extremely powerful for summarising business data.

```
sales = pd.DataFrame({
    "Region": ["North", "South", "North", "South"],
    "Sales": [1000, 1500, 2000, 1200]
})
print(sales.groupby("Region")["Sales"].sum())
```

Output:

```
Region
North    3000
South    2700
Name: Sales, dtype: int64
```

Business Dataset Example — Sales Report Analysis

```
sales = pd.DataFrame({
    "Product": ["Laptop", "Mouse", "Laptop", "Keyboard", "Mouse"],
    "Units": [5, 20, 3, 15, 10],
    "Price": [50000, 500, 50000, 1500, 500]
})
sales["Revenue"] = sales["Units"] * sales["Price"]
summary = sales.groupby("Product")["Revenue"].sum().sort_values(ascending=False)
print(summary)
```

Line	Explanation
6	Create a new Revenue column by multiplying Units and Price for every row.
7	Group rows by Product, sum their Revenue, then sort the result from highest to lowest.

Output:

```
Product
Laptop    400000
Keyboard   22500
Mouse     15000
Name: Revenue, dtype: int64
```

Interview Questions — Module 18

Q1. What is the difference between a Pandas Series and a DataFrame?

Ans: A Series is a one-dimensional labelled array (like a single column). A DataFrame is a two-dimensional table made up of multiple Series sharing a common index (like a full spreadsheet with rows and columns).

Q2. How do you handle missing data in Pandas?

Ans: Detect it with `isnull()`, then either remove affected rows with `dropna()` or replace missing values with a sensible default using `fillna()`, depending on whether the missing data can be safely dropped or should be estimated.

Q3. What does `groupby()` do?

Ans: It splits a `DataFrame` into groups based on the unique values of one or more columns, so an aggregate function (like `sum`, `mean`, or `count`) can be applied separately to each group — the 'split-apply-combine' pattern.

Q4. What is the difference between `loc[]` and `iloc[]`?

Ans: `loc[]` selects rows/columns by their label/name (e.g., `df.loc['Riya']`). `iloc[]` selects by integer position (e.g., `df.iloc[0]`), regardless of the actual index labels.

Q5. How would you filter rows in a `DataFrame` based on a condition?

Ans: Using boolean indexing, e.g., `df[df['Marks'] > 75]` returns only the rows where the `Marks` column exceeds 75.

MCQs — Module 18

No.	Question	Answer	Options (A-D)
1	A Pandas Series is:	A	A) 1-D labelled array B) 2-D table C) A CSV file D) A NumPy array only
2	Which function reads a CSV file into a <code>DataFrame</code> ?	C	A) <code>pd.load_csv()</code> B) <code>pd.open_csv()</code> C) <code>pd.read_csv()</code> D) <code>pd.csv()</code>
3	Which method removes rows with missing values?	B	A) <code>fillna()</code> B) <code>dropna()</code> C) <code>removen()</code> D) <code>clean()</code>
4	<code>df.groupby('X')['Y'].sum()</code> computes:	D	A) Total of all rows B) Average of X C) Count of Y D) Sum of Y for each group in X
5	Which selects rows by label, not position?	A	A) <code>loc[]</code> B) <code>iloc[]</code> C) <code>at[]</code> D) <code>get[]</code>
6	<code>df.head()</code> by default shows:	C	A) All rows B) Last 5 rows C) First 5 rows D) Column names only

Exercises — Module 18

1. Create a `DataFrame` of 5 employees with `Name`, `Department`, and `Salary`, then find the average salary per department.
2. Load a CSV file and print the number of missing values in each column.
3. Filter a `DataFrame` of products to show only those with `Price > 1000`.
4. Write a program to remove duplicate rows from a `DataFrame`.
5. Use `groupby` to find the total sales per region from a sales `DataFrame`.

MODULE 19: MATPLOTLIB

Matplotlib is Python's foundational plotting library, used to turn raw numbers into visual charts — essential for spotting trends, comparing categories, and communicating data findings clearly.

Setup

Install once with: `pip install matplotlib` Then import with: `import matplotlib.pyplot as plt`

19.1 Line Chart

Best for showing a trend over a continuous range, such as time.

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [200, 250, 180, 300, 280]

plt.plot(months, sales, marker="o")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales (units)")
plt.show()
```

```
Sales
300 |
280 |
250 |
200 |
180 |
    +-----+
      Jan Feb Mar Apr May
(Illustrative sketch of the resulting line chart)
```

19.2 Bar Chart

Best for comparing distinct categories against each other.

```
products = ["Laptop", "Mouse", "Keyboard"]
units_sold = [45, 120, 80]

plt.bar(products, units_sold, color="steelblue")
plt.title("Units Sold by Product")
plt.show()
```

19.3 Pie Chart

Best for showing each category's share of a whole (percentages that add up to 100%).

```
labels = ["Marketing", "R&D", "Sales", "Support"]
budget = [30, 25, 35, 10]

plt.pie(budget, labels=labels, autopct="%1.1f%%")
plt.title("Budget Allocation")
plt.show()
```

Common Beginner Mistake

Using a pie chart for more than 5-6 categories makes it cluttered and hard to read. For many categories, a bar chart is almost always clearer.

19.4 Histogram

Best for showing the distribution/frequency of a single numeric variable, grouped into ranges (bins).

```
scores = [55, 62, 70, 72, 75, 78, 80, 85, 88, 90, 92, 95]

plt.hist(scores, bins=5, color="seagreen", edgecolor="black")
plt.title("Distribution of Exam Scores")
plt.xlabel("Score Range")
plt.ylabel("Number of Students")
plt.show()
```

19.5 Scatter Plot

Best for showing the relationship (correlation) between two numeric variables.

```
hours_studied = [1, 2, 3, 4, 5, 6, 7]
marks_scored = [35, 40, 55, 60, 70, 82, 90]

plt.scatter(hours_studied, marks_scored, color="crimson")
plt.title("Study Hours vs Marks Scored")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.show()
```

Choosing the Right Chart Type

Chart Type	Best For	Avoid When
Line	Trends over time/continuous data	Comparing unrelated categories
Bar	Comparing distinct categories	Showing trend over time
Pie	Share of a whole, few categories	Many categories (>6) or precise comparison
Histogram	Distribution of one numeric variable	Comparing categories, not ranges
Scatter	Relationship between two numeric variables	Categorical (non-numeric) data

Visualization Project — Sales Dashboard (Multiple Subplots)

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].plot(months, sales, marker="o")
axes[0].set_title("Sales Trend")

axes[1].bar(products, units_sold, color="orange")
axes[1].set_title("Units by Product")

plt.tight_layout()
plt.show()
```

subplots() lets you display multiple charts side by side in one figure — very common in real dashboards.

Interview Questions — Module 19

Q1. When would you use a line chart instead of a bar chart?

Ans: A line chart is best for showing a trend over a continuous sequence, like time (e.g., monthly sales). A bar chart is best for comparing distinct, separate categories against each other.

Q2. What does plt.show() do, and why is it needed?

Ans: It renders and displays the figure that has been built up with the preceding plotting commands. Without calling it, the plot may not appear (especially outside interactive environments like Jupyter).

Q3. When is a pie chart NOT a good choice?

Ans: When there are more than 5-6 categories, or when precise comparisons between similar-sized slices matter — human eyes are poor at comparing angles/areas accurately; a bar chart communicates this better.

Q4. What is the purpose of a histogram, and how does it differ from a bar chart?

Ans: A histogram shows the distribution of a single continuous numeric variable by grouping values into ranges (bins). A bar chart compares distinct, separate categories — a histogram's bins are numeric ranges, not independent categories.

Q5. How would you display two charts side by side in the same figure?

Ans: Using plt.subplots(rows, cols) to create a grid of axes, then plotting on each axes object individually (e.g., axes[0].plot(...), axes[1].bar(...)).

MCQs — Module 19

No.	Question	Answer	Options (A-D)
1	Which chart is best for showing a trend over time?	B	A) Pie chart B) Line chart C) Histogram D) Scatter plot
2	Which function creates a bar chart?	C	A) plt.line() B) plt.pie() C) plt.bar() D) plt.hist()
3	Which chart shows the relationship between two numeric variables?	D	A) Bar B) Pie C) Histogram D) Scatter
4	Which function actually displays the plot?	A	A) plt.show() B) plt.display() C) plt.render() D) plt.plot()

No.	Question	Answer	Options (A-D)
5	A histogram is used to show:	B	A) Category comparison B) Distribution of a numeric variable C) Two variables' relationship D) Percentage share
6	Which is generally avoided for more than 6 categories?	C	A) Bar chart B) Line chart C) Pie chart D) Scatter plot

Exercises — Module 19

1. Plot a line chart showing your own weekly study hours over 7 days.
2. Create a bar chart comparing the population of 5 cities.
3. Create a pie chart showing how you spend a 24-hour day (sleep, study, leisure, etc.).
4. Create a histogram of 20 random exam scores.
5. Create a scatter plot showing the relationship between advertising spend and sales for 10 data points.

QUICK REVISION NOTES — MODULES 17-19

- NumPy's core object is the ndarray — fast, same-typed, supports vectorised (loop-free) elementwise operations.
- Key NumPy functions: `np.array()`, `.shape`, `.sum()`/`.mean()`/`.max()`/`.min()`, `.reshape()`, boolean indexing (`arr[arr>x]`).
- Pandas Series = 1-D labelled column; DataFrame = 2-D table of multiple Series sharing an index.
- Clean data with `isnull()`, `dropna()`, `fillna()`; filter rows with boolean indexing `df[df['col']>x]`.
- `groupby('col')['target'].agg_func()` implements the split-apply-combine pattern for summarising data by category.
- Matplotlib chart choice: Line=trend over time, Bar=category comparison, Pie=share of a whole (few categories), Histogram=distribution of one variable, Scatter=relationship between two variables.
- `plt.show()` renders the figure; `plt.subplots()` creates multiple charts in one figure.

COMMON ERRORS AND FIXES — MODULES 17-19

Error	Cause	Fix
ModuleNotFoundError: numpy/pandas/matplotlib	Library not installed	Run: <code>pip install numpy pandas matplotlib</code>
ValueError: cannot reshape array	Target shape doesn't match total element count	Ensure <code>rows × columns</code> = original number of elements
KeyError on DataFrame column	Misspelled or non-existent column name	Check exact spelling via <code>df.columns</code>
Plot window doesn't appear	Forgot to call <code>plt.show()</code>	Always end plotting code with <code>plt.show()</code>
SettingWithCopyWarning in Pandas	Modifying a filtered slice of a DataFrame	Use <code>df.copy()</code> when creating a filtered subset you plan to edit

CODING BEST PRACTICES — MODULES 17-19

- Prefer vectorised NumPy/Pandas operations over manual Python loops — they're faster and more idiomatic.
- Always inspect new data first with `df.head()`, `df.info()`, and `df.describe()` before analysing it.
- Handle missing data deliberately (drop or fill) — don't silently ignore `isnull()` warnings.
- Always label your charts: title, xlabel, ylabel — an unlabelled chart is hard to interpret later.
- Choose the chart type that matches your data's story (see the chart-selection table) rather than defaulting to the same chart every time.